

함수형 프로그래밍

학습 체크리스트

- ☐ 함수형 프로그래밍 패러다임의 역사와 개념 이해
- ☐ 자바 랴다(Lambda)와 함수형 인터페이스 활용
- ☐ 스트림(Stream) API의 동작 메커니즘과 JS와의 차이점
- ☐ 스트림 체이닝의 지연 연산 및 성능 최적화 구조 분석
- ☐ 병렬 스트림(Parallel Stream)의 동작 원리와 부작용 인지

함수형 프로그래밍 패러다임

함수형 프로그래밍(Functional Programming)

프로그래밍을 수학적 함수의 계산으로 취급하고 상태 변경과 가변 데이터를 피하려는 선언형 프로그래밍 패러다임

- 선언형 패러다임 지향: 어떻게 할 것인가(How)보다 무엇을 할 것인가(What)를 기술함
- 가변 상태를 회피하는 이유:
 - 스레드 안전성: 여러 스레드가 동시에 접근하더라도 경쟁 상태(Race Condition)를 원천 차단함
 - 부작용 제거: 연산 과정이 외부 변수의 값을 침범하지 않아 코드의 오작동과 결함을 줄임
 - 참조 투명성: 동일 입력에 대해 항상 동일 결과를 반환하여 동작 신뢰성과 테스트 가독성을 극대화함

함수형 프로그래밍의 역사적 계보 (1/2)

- 리스프(Lisp)
 - 1958년 존 매카시가 개발한 실질적인 함수형 프로그래밍의 시조
 - 코드와 데이터를 동일한 리스트 구조로 표현하는 동형성을 가짐
 - 일급 함수(First-class function)와 가비지 컬렉션(GC) 개념을 최초로 대중화함
- 하스켈(Haskell)
 - 부작용(Side Effect)을 철저히 배제하는 **순수 함수형 언어**(Pure Functional)의 표준
 - 동일 입력에 대해 항상 동일 출력을 보장하는 **참조 투명성**을 유지함
 - 지연 평가(Lazy Evaluation)와 정적 타입 기반 모나드(Monad)로 부작용을 관리함

함수형 프로그래밍의 역사적 계보 (2/2)

- 스칼라(Scala)
 - JVM 환경에서 객체지향(OOP)과 함수형(FP)의 이상적인 결합을 이룬 하이브리드 언어
 - 자바 코드와의 완벽한 상호운용성(Interop)을 유지하면서 선진적 함수형 구조를 제공함
 - 패턴 매칭, 불변 데이터 구조(case class), 표현력 높은 타입 시스템 도입
 - 자바가 JDK 8 이후 현대적 다중 패러다임 언어로 진화하는 데 영감을 제공함

Java의 함수형 패러다임 도입 배경

- 멀티코어와 데이터의 팽창: 대규모 데이터 처리 및 멀티코어 환경에서의 효과적인 병렬 처리가 절실했음
- 패러다임의 융합: 기존의 탄탄한 객체지향(OOP) 구조 위에 함수형(FP) API를 자연스럽게 이식함 (JDK 8+)
- 부작용의 억제: 외부 상태 변경을 피하는 순수 함수와 불변성(Immutability)을 통해 동시성 환경에서의 스레드 안전성을 확보함

함수형 인터페이스

함수형 인터페이스(Functional Interface)

단 하나의 추상 메서드(Single Abstract Method, SAM)만을 정의하여 람다식의 타겟 타입이 되는 인터페이스

- 컴파일 시점 검증: `@FunctionalInterface` 어노테이션을 부착하여 단일 메서드 규칙 위반을 방지함
- 다양한 표준 규격: 표준화된 동작 흐름을 지원하기 위해 `java.util.function` 에 핵심 규격들을 내장함

람다식과 메서드 참조

람다식(Lambda Expression)

메서드를 하나의 간결한 식으로 표현하여 행위 자체를 값으로 취급하고 전달할 수 있게 하는 익명 함수

- **변수 캡처링 제약**: 람다 내부에서 외부 지역 변수를 읽을 때 해당 변수는 `final` 또는 `effectively final` 상태여야 함
- **메서드 참조(Method Reference)**: 람다가 단순히 기존 메서드를 위임 호출하는 형태라면 `클래스명::메서드명`으로 간소화함

메서드 참조의 축약 표현

```
1 // 1. 정적 메서드 참조 (Static Method Reference)
2 Function<String, Integer> f1 = s → Integer.parseInt(s);
3 Function<String, Integer> f2 = Integer::parseInt;
4
5 // 2. 인스턴스 메서드 참조 (Instance Method Reference)
6 Consumer<String> c1 = s → System.out.println(s);
7 Consumer<String> c2 = System.out::println;
```

Comparator 체이닝을 통한 정렬 최적화

- 우아한 다차원 정렬: 자바 디폴트 메서드를 결합해 번잡한 익명 구현 객체 없이 복합 정렬 구조를 기술함
- 가독성 향상: `Comparator.comparing` 및 `thenComparing` 을 연결해 체계적인 정렬 기준의 계층을 나타냄
- 타입 추론: 도메인 객체의 Getter 메서드 참조를 통해 정렬 키 타입을 안전하게 자동 추론함

정렬 대상 도메인 클래스 설계

```
1  class User {
2      private final String name;
3      private final int age;
4
5      public User(String name, int age) { this.name = name; this.age = age; }
6      public String getName() { return name; }
7      public int getAge() { return age; }
8      public String toString() { return name + "(" + age + ")"; }
9  }
```

Comparator 다차원 정렬 수행

```
1 List<User> users = new ArrayList<>(List.of(
2     new User("Kim", 25), new User("Lee", 30), new User("Kim", 20)
3 ));
4
5 // 이름순으로 1차 정렬한 뒤, 이름이 같으면 나이 오름차순으로 2차 정렬
6 users.sort(
7     Comparator.comparing(User::getName)
8         .thenComparingInt(User::getAge)
9 );
10 System.out.println(users); // [Kim(20), Kim(25), Lee(30)]
```

스트림 데이터 파이프라인의 이해

- **자재 창고와 가공 컨베이어 벨트**
 - 컬렉션(Collection)이 자재들이 쌓여 있는 원자재 창고라면, 스트림(Stream)은 이 자재들을 하나씩 컨베이어 벨트에 흘려보내는 흐름선임
- **지연된 스위치**
 - 중간 가공 장치(filter, map)를 설치하더라도 포장 완료 단계(최종 연산) 스위치를 작동시키지 않으면 벨트는 일절 흐르지 않음
- **수요 기반의 개별 흐름**
 - 포장 박스에 원자재가 인입되는 요청 시점에 비로소 요소가 파이프라인에 주입되어 최적화된 처리를 수행함

Java Stream과 JavaScript 고차함수

- **공통 분모:** 원본 데이터를 훼손하지 않으면서 순수 함수 연산들을 엮어 가독성 높은 비즈니스 로직을 구축함
- **JS의 즉시 연산(Eager Evaluation)**
 - 배열 메서드(`map`, `filter`)를 호출할 때마다 매 단계마다 임시 가공 배열을 즉각 새로이 생성해 냄
- **Java의 지연 연산(Lazy Evaluation)**
 - 최종 연산이 격발되기 전까지는 실제로 요소를 꺼내지 않고 선언된 파이프라인 명세서만 결합하여 들고 있음
- **진입 방식:** JS는 배열 객체에서 즉각 체이닝이 가능하나, 자바는 컬렉션을 `.stream()` 을 거쳐 전용 객체로 치환해야 함

Java 핵심 표준 함수형 인터페이스

- `Consumer<T>`: 인자 T를 전달받아 이를 소비하고 반환하지 않음 (`void accept(T t)`)
- `Supplier<T>`: 인자를 받지 않고 필요한 결과값 T를 동적으로 생성해 제공함 (`T get()`)
- `Function<T, R>`: 인자 T를 받아 비즈니스 로직을 거쳐 R 타입 객체로 변환해 매핑함 (`R apply(T t)`)
- `Predicate<T>`: 인자 T를 평가해 정합성 여부를 판단하고 참/거짓 논리값을 반환함 (`boolean test(T t)`)
- `BinaryOperator<T>`: 동일한 타입의 두 인자 T를 받아 연산 후 다시 같은 타입 T로 반환함 (`T apply(T t1, T t2)`)

언어별 데이터 가공 연산 대조

Java Stream 메서드	JavaScript Array 메서드	설명	연산 종류
<code>filter(Predicate)</code>	<code>filter(Callback)</code>	조건에 맞는 요소만 추출	중간 연산
<code>map(Function)</code>	<code>map(Callback)</code>	요소를 다른 값이나 타입으로 변환	중간 연산
<code>reduce(BinaryOperator)</code>	<code>reduce(Callback)</code>	누적 연산을 통해 최종 단일 값 도출	최종 연산
<code>forEach(Consumer)</code>	<code>forEach(Callback)</code>	각 요소를 순회하며 소비적인 행위 수행	최종 연산

JavaScript의 즉시 데이터 매핑

```
1 // 호출 직후 filter와 map에 해당하는 임시 가공 배열이 순차적으로 메모리에 매번 올라감
2 const result = [1, 2, 3, 4, 5]
3   .filter(n => n % 2 !== 0)
4   .map(n => n * 2);
5
6 console.log(result); // [2, 6, 10]
```

Java Stream의 지연 데이터 파이프라인

```
1 // collect() 최종 연산이 트리거되는 시점에 맞추어 연산 처리가 본격적으로 수행됨
2 List<Integer> list = List.of(1, 2, 3, 4, 5);
3 List<Integer> result = list.stream()
4     .filter(n → n % 2 ≠ 0)
5     .map(n → n * 2)
6     .collect(Collectors.toList());
7
8 System.out.println(result); // [2, 6, 10]
```

중간 연산과 최종 연산의 특징

- 중간 연산 (Intermediate)
 - 가공된 새로운 스트림 객체를 반환하므로 체이닝 방식으로 무한히 덧붙여 선언할 수 있음
 - 실제로 연산을 구동하지 않고 파이프라인 경로 설계 정보만 연쇄 보존함 (`filter`, `map` 등)
- 최종 연산 (Terminal)
 - 가공 명세에 따라 요소를 실질적으로 소모해 최종 데이터 타입 형태로 변환하며 파이프라인을 완전히 종료함
 - 한 번 최종 연산이 완료된 스트림은 재생성하여 다시 사용할 수 없음 (`collect`, `reduce` 등)

스트림 파이프라인의 최적화 기법

- 루프 퓨전(Loop Fusion)
 - 각각의 연산을 돌기 위해 매번 루프를 순차적으로 도는 게 아님
 - 하나의 요소가 중간 연산 A, B를 연속해서 거치고 최종 연산에 직행한 뒤 다음 요소가 출발하는 수직적 통합 구조임
- 쇼트 서킷(Short-circuit)
 - 스트림 파이프라인 중간에 조기 단락 조건을 충족하면, 뒷열의 요소들이 남아있더라도 불필요한 연산을 완전히 생략하고 즉시 종료함 (`limit`, `findFirst`)

수직 평가와 쇼트 서킷의 가동 과정

```
1 List<String> names = List.of("Kim", "Park", "Lee", "Choi");
2 names.stream()
3     .filter(n → { System.out.println("F: " + n); return n.length() ≥ 4; })
4     .map(n → { System.out.println("M: " + n); return n.toUpperCase(); })
5     .limit(1) // 하나의 데이터가 매핑 완료되면 Lee, Choi는 필터조차 타지 않음
6     .forEach(System.out::println);
```

병렬 스트림의 개념과 구동

병렬 스트림(Parallel Stream)

ForkJoinPool을 백엔드로 활용해 대용량 데이터소스를 조각으로 분할하고 멀티 코어 환경에서 병렬로 연산을 대행하는 스트림

- **간편한 선언:** 일반 스트림 선언 뒤에 `.parallel()` 을 덧붙이거나 컬렉션에서 `parallelStream()` 을 즉시 받아 사용함
- **분할 정복:** 공통의 데이터 청크를 하위 단위로 쪼갬 뒤 작업 스레드가 각자 연산해 모으는 방식으로 최적화를 꾀함

병렬 스트림 사용 시 성능적 한계와 오버헤드

- 구조적 분할의 난이도: 데이터소스가 `ArrayList` 처럼 주소 균등 배분이 가능하면 빠르나, 스캔해야 하는 `LinkedList` 인 경우 쪼개는 오버헤드가 더 큼
- 작업 분할 및 통합 비용: 스레드 간 일감을 쪼개고(Split) 다 끝나면 다시 복원 취합(Merge)하는 일련의 과정에 추가 비용이 들어감
- 컨텍스트 스위칭: 연산 강도가 극도로 얇은 데이터라면 오히려 스레드 간 전환 비용이 순차 싱글 스레드 처리보다 느려지게 만들

병렬 스트림 사용 시 주의해야 할 2대 치명적 위험

- **공유 가변 상태의 오염:** 전달하는 람다식 내부에 외부 공유 변수(Mutable State)를 조작하는 행위가 있다면 경쟁 상태에 빠져 데이터가 망가짐
- **공통 스레드 풀의 마비와 오염 (CommonPool Block)**
 - 병렬 스트림은 CPU 코어 수에 비례하는 고정 크기의 전역 스레드 풀(`ForkJoinPool.commonPool()`)을 공유함
 - 파이프라인 내부에서 블로킹 I/O(네트워크 요청, DB 조회 등)를 타면 스레드가 대기 상태로 고갈되어 풀 자체가 차단됨
 - 이로 인해 동일 스레드 풀을 사용하는 애플리케이션 내의 모든 병렬 작업이 전면 지연되는 연쇄 장애로 번짐

비안전 공유 컬렉션 쓰기 시 경쟁 상태

```
1 List<Long> result = new ArrayList<>();
2 LongStream.rangeClosed(1, 100_000)
3     .parallel()
4     .forEach(result::add); // ArrayList는 멀티스레드 정합성을 보장하지 않음!
5
6 System.out.println("예상 개수: 100000");
7 System.out.println("실제 저장된 개수: " + result.size());
8 // 스레드 경합으로 인해 누락이 유발되어 100,000보다 현저히 적은 수치가 나옴
```

핵심 정리: 함수형 프로그래밍과 스트림

- 함수형 패러다임 지향: 불변성과 순수 함수를 활용해 스레드 안전성을 보장하고 부작용(Side Effect)을 원천 통제함
- 스트림 파이프라인 최적화: 최종 연산 전까지 가공 명세만 수립하며, 루프 퓨전과 쇼트 서킷을 통해 효율을 극대화함
- 병렬 스트림의 제약: 공통 스레드 풀(`commonPool`) 오염 위험성 때문에 블로킹 I/O 작업을 절대 결합해서는 안 됨
- 올바른 도입 판단: 분할하기 쉬운 ArrayList 구조이거나, 개별 요소당 연산 강도가 높아 병렬화 비용을 상쇄할 때 선택함